

Projekt Ewolucja Otwarta

Filip Mieszkowski i Stanisław Kurzątkowski¹

¹Politechnika Warszawska, Warszawa, Polska.

Streszczenie

Projekt polega na symulowaniu ewolucji otwartej oraz próbie zaobserwowania emergencji pewnych zjawisk znanych z ewolucji biologicznej. Projekt wykorzystuje kartę graficzną, aby zapewnić dużą moc obliczeniową potrzebną do symulacji dużych populacji

1 Reguły symulacji

W tej sekcji wymiennie będziemy stosowali słowa **osobnik** i **agent**. Stałe symulacji mogą być zmieniane, jednakże w nawiasach będziemy podawali ich liczbową wartość, której użyliśmy w eksperymentach.

1.1 Mapa

Symulacja odbywa się na płaskiej, dyskretniej płaszczyźnie kwadratowej (rozmiaru 256), której krawędzie domknięte są w kształt torusa (podobnie jak np. w grze Snake). Na każdym polu może znajdować się jeden lub więcej agentów, jedzenie (którego liczba wyraża się dodatnią liczbą rzeczywistą), woda (działająca identycznie jak jedzenie, stanowiąca jednak oddzielny zasób) a także poziom zagrożenia.

W każdym kroku symulacji pewna ilość jedzenia i wody pojawia się na mapie. Dzieje się to jednak w sposób nieregularny zarówno w czasie jak i w przestrzeni. Warunki symulacji są zmienne w czasie, a liczba jedzenia i wody dodawanych w kroku t symulacji wyraża się wzorem:

$$\text{round} \left(Q \cdot \left(S_o + S_a \cdot \sin \left(\frac{2\pi t}{P} \right) \right) \right)$$

gdzie Q to bazowa ilość zasobów (1024), S_o to stałe przesunięcie pory roku (1), S_a to amplituda zmian sezonowych (0.5), a P to długość okresu pory roku (500). Dodatkowo

jedzenie i woda nie jest rozłożone równo w przestrzeni. Na mapie znajduje się pewna liczba (2) **rzek** jedzenia wody. Są to długie i wąskie obszary, na których pojawia się 90% zasobów. Rzeki jedzenia i wody nie pokrywają się ze sobą, jednakże w kilku miejscach się krzyżują tworząc **obszary obfitości** (nazywane dalej również **oazami**).

1.2 Agenci

Każdy agent ma dostępne pewne sensory, akcje, które może wykonywać a także ośrodek decyzyjny będący siecią neuronową. Żaden z tych trzech komponentów nie zmienia się w trakcie życia agenta (dotyczy to również wag sieci neuronowej), jest natomiast modyfikowany w sposób losowy w wyniku mutacji, w momencie narodzin nowego osobnika (reprodukcji).

1.2.1 Sensory

Każdy agent symulacji ma pewną liczbę (62) sensorów zewnętrznych. Każdy z tych sensorów ma swoje współrzędne określone dwoma liczbami całkowitymi (określającymi, na które pole symulacji względem agenta działa dany sensor), a także swój typ określający co dany sensor wykrywa. Typy sensorów pokrywają się z obiektami, które mogą znajdować się na mapie (agenci, jedzenie, woda, zagrożenie). Dodatkowo każdy agent ma (niepodlegające mutacji) dwa sensory określające cosinus i sinus obecnej pory roku.

1.2.2 Akcje

Każdy agent ma do wyboru pewną liczbę (32) akcji. Każda akcja (podobnie jak sensor) ma swoje współrzędne (pole, na które dana akcja działa względem położenia agenta) a także typ, określający co dana akcja robi. Dostępne są możliwe typy akcji: spożycie jedzenia, spożycie wody, umieszczenie zagrożenia, ruch (na dane pole) oraz reprodukcja.

1.2.3 Wewnętrzne sieci neuronowe

Agenci podejmują decyzje o akcji na podstawie odczytów z sensorów poprzez ich wewnętrzne sieci neuronowe. Kształt sieci jest zadany przez symulację, składa się on z dwóch warstw gęstych (z warstwą ukrytą o zadanym rozmiarze (32)). Na ostatniej (wyjściowej) warstwie zamontowany jest softmax, który zadaje prawdopodobieństwa danej akcji. Neurony wejściowe dokładnie pokrywają się z sensorami (których łącznie jest 64), zaś neurony wyjściowe dokładnie pokrywają się z akcjami, które agent może wykonać.

1.2.4 Jedzenie, woda i śmierć

Każdy osobnik ma swój poziom jedzenia i poziom wody, które zmniejszają się w każdym kroku symulacji o zadaną wartość (0.02). Jeżeli jeden z tych poziomów spadnie do 0, to osobnik umiera. Osobniki mogą zwiększać poziom jedzenia i wody za pomocą akcji spożywania jedzenia i wody. Otrzymują wówczas odpowiedni zasób w liczbie, w jakiej ten zasób znajdował się na danym polu.

1.2.5 Reprodukacja i mutacja

Jeżeli osobnik wybrał akcję **reprodukacja**, to (o ile spełnia warunek reprodukowalności, tzn. zarówno jego poziom jedzenia jak i poziom wody jest większy niż pewna stała wartość (0.6)) może powołać do życia nowego osobnika. Ponosi przy tym jednak pewien dodatkowy koszt jedzenia i wody (0.2), a dodatkowo przekazuje swojemu potomkowi pewną liczbę swojego jedzenia i wody (połowę). Nowy osobnik dziedziczy (z mutacją) po swoim rodzicu wszystkie akcje, sensory a także wartości wag wewnętrznej sieci neuronowej. Mutacja poszczególnych cech przebiega następująco: 0, 1 lub 2 sensory (losowo) są usuwane, a w ich miejsce losowane są zupełnie nowe (z nowymi współrzędnymi i typem sensora), wszystkie wagi sieci neuronowej są modyfikowane o wartość losową z zadanego rozkładu $\mathcal{N}(0, 0.2)$. 0, 1, lub dwie akcje są usuwane i zastępowane nowymi (podobnie jak sensory).

1.2.6 Zagrożenie i walka

Agent może wybrać akcję **walka** i wówczas na zadanym polu umieszcza pewną ilość **zagrożenia** (3). Jeżeli w tym samym kroku symulacji na tym polu znajduje się inny agent (albo nawet ten sam, jeżeli umieścił zagrożenie na swoim polu), to traci on tyle jedzenia i wody ile wynosi zagrożenie na tym polu (czyli potencjalnie całe jedzenie i wodę), zaś ilość jedzenia i wody, jaką stracił pojawia się na polu (i może zostać spożyta).

2 Etapy rozwoju projektu

Rozwój projektu można podzielić na dwa główne etapy. W tej sekcji opisane zostaną cele każdego etapu, wnioski wyciągnięte po jego zakończeniu i modyfikacje wykonane na podstawie wniosków. Sekcja **1 Reguły symulacji** opisuje stan końcowy projektu.

2.1 Etap I - Implementacja wersji podstawowej

Założeniem tego etapu było stworzenie implementacji, która obejmuje podstawowe akcje agentów oraz ich interakcje z otoczeniem i między sobą nawzajem. Wyznacznikiem świadczącym o ukończeniu tego etapu było potwierdzenie, że symulacja działa poprawnie i nie zawiera błędów implementacji lub logicznych, które w wyraźny sposób zaburzałyby ewolucję układu.

2.1.1 Cele do zaimplementowania

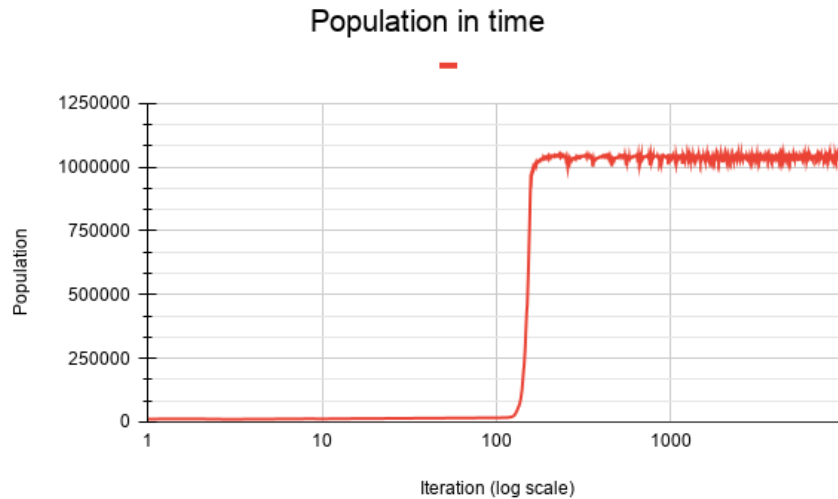
Poziom zaawansowania symulacji na końcu tego etapu obejmuje:

1. Prostokątna mapa z geometrią torusa i z 3 warstwami złożoności: FOOD, DANGER, CREATURE
2. Agenci reprezentowani przez sieć MLP z jedną warstwą ukrytą, biasem i funkcją aktywacji ReLU
3. Sensory agentów wykrywające: FOOD, DANGER, CREATURE
4. Akcje możliwe do podjęcia przez agentów: MOVE, EAT, ATTACK, REPRODUCE
5. Pasywne tempo losowego pojawiania się FOOD na mapie

2.1.2 Testy

Po zaimplementowaniu pierwszego etapu działania symulacji zostało przetestowane. Jeden z pierwszych, długich (>10 000 iteracji) zapisanych przebiegów, który spełniał wszystkie założenia **2.1.1** dostępny jest do obejrzenia [tutaj](#).

Kolory: zielony - FOOD, czerwony - DANGER, niebieski - CREATURE, magenta - DANGER + CREATURE



Rysunek 1 Wykres przedstawiający podejrzanie szybko rosnącą w czasie populację, która następnie utrzymuje się na maksymalnym poziomie (narzuconym sztucznie, programistycznie w celu zabezpieczenia pamięci).

2.1.3 Wnioski

Wizualizacja przebiegu symulacji w połączeniu z wykresem populacji pozwoliła stwierdzić, że przebieg symulacji odbiega od oczekiwanego i najprawdopodobniej zawiera pewien błąd. Początkowo stwierdzono:

1. Agenci kumulują się w dużą kolonię i nie eksplorują mapy
2. Etap symulacji jest zbyt wczesny by takie zjawisko miało uzasadnienie w procesie ewolucji
3. Obserwowana kolonia jest bardzo "gęsta", skupia prawie całą populację miliona osobników na małym obszarze
4. Wewnątrz obszaru koloni bardzo często wykonywana jest akcja ataku, co powinno skutkować bardzo dużą śmiertelnością i wyludnieniem obszaru, jednakże tak się nie dzieje, a kolonia nadal trwa
5. Ilość pojawiającego się na mapie pożywienia jest ograniczona i nie powinna pozwalać na utrzymanie tak dużej populacji

W celu dalszej diagnozy, pliki binarne map zapisywane w trakcie symulacji w odstępach 10 iteracji zostały wykorzystane do sporządzenia wizualizacji z rozdzieleniem na poszczególne warstwy mapy. Dwa pliki wizualizacji dla przebiegu z tym samym zjawiskiem [tutaj](#).

Zaobserwowano:

1. Obszar zgromadzenia osobników jest bardzo obfity w pożywienie
2. Pożywienie to musi pochodzić z akcji ataku wykonywanej przez osobniki, która zabijając inne osobniki, skutkuje pojawieniem się na mapie dodatkowych jednostek jedzenia

Postanowiono przyjrzeć się ówczesnej mechanice ataku, której logika była następująca:

1. Atak zawsze odbiera atakowanemu agentowi 1.0 jednostki jego wewnętrznego poziomu energii, co symbolizuje pogorszenie się jego stanu
2. Na skutek ataku i otrzymania obrażeń, zaatakowany agent upuszczał pewną ilość *biomasy* w postaci FOOD, której wartość wynosiła $0.8 \cdot agent_internal_energy$

Proste obliczenia wskazują, że jeżeli atakowany agent ma wysoki poziom jedzenia (np. powyżej 1.25), to w wyniku zaatakowania go w symulacji powstaje jedzenie (osobnik traci mniej jedzenia, niż pojawia się na polu). Ten drobny błąd programistyczny został wykorzystany przez agentów do gwałtownego rozrostu (a w zasadzie „wybuchu”) populacji. Pomimo, że nie to było naszym celem, to fakt, że agenci byli w stanie wykorzystać ten błąd do budowy „farmy” uznajemy za dość imponujący.

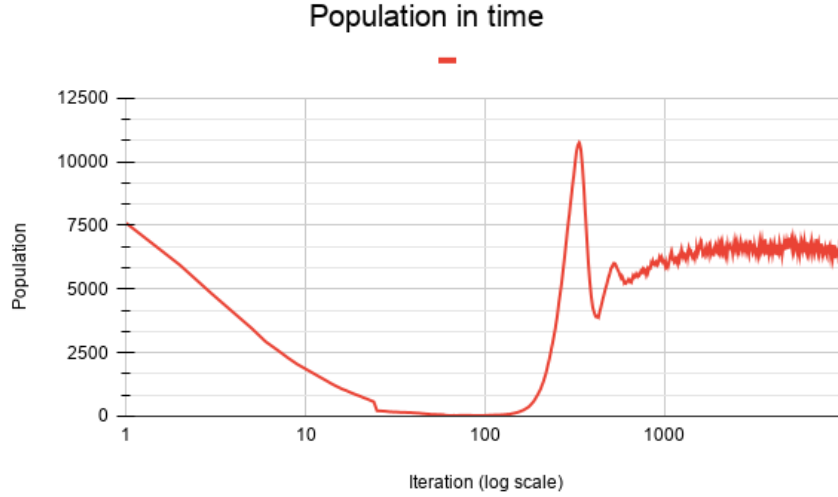
2.1.4 Modyfikacje

Postanowiono zmienić sposób działania mechaniki ataku. Głównym motywem wprowadzanych zmian była **Zasada Zachowania Energii**. Akcja ataku nie może generować energii, powinna ona jedynie umożliwiać przechwycenie porcji energii pomiędzy agentami. Zdecydowano, że akcja ta musi podlegać następującym regułom:

1. Atak musi mieć swój, stosunkowo wysoki koszt akcji *ATTACK_COST* oraz stosunkowo dużą skuteczność *ATTACK_DAMAGE*
2. Atak skutkuje odebraniem agentowi atakowanemu *ATTACK_DAMAGE* od jego poziomu energii
3. Jeśli atak nie zabija atakowanego agenta, na mapie nie pojawia się FOOD
4. Jeśli atak zabija atakowanego agenta, w jego miejsce pojawia się FOOD o wartości jego wewnętrznego poziomu energii w momencie śmierci

Taka mechanika ataku ma być bardziej wymagająca w stosowaniu oraz nie powodować generowania nowej energii w systemie. Po zaimplementowaniu tej logiki w następnych przebiegach symulacji już nigdy nie zaobserwowano zjawiska nadużywania akcji ataku. Przykładowy przebieg symulacji [tutaj](#).

Początkowa populacja ustawiona parametrem symulacji na 7500 osobników prawie całkowicie obumiera, przy życiu pozostaje dokładnie 25 agentów posiadających podstawowe przystosowanie do środowiska, ze względu na skumulowany w środowisku pokarm, widoczny jest dynamiczny rozrost populacji, aż powyżej progu pojemności



Rysunek 2 Wykres przedstawia oczekiwany przebieg liczności populacji w czasie.

środowiska. Następstwem jest obumierania nadmiaru populacji i lekkie oscylacje do stanu równowagi układu, znajdującego się lekko poniżej maksymalnej pojemności środowiska.

Maksymalną pojemność środowiska można wyznaczyć na podstawie wspomnianego w sekcji 1.2.4 pasywnego kosztu życia ponoszonego przez każdego żywego osobnika w każdej iteracji symulacji oraz ilości pasywnie pojawiającego się na mapie jedzenia. Możemy napisać równanie bilansu energetycznego z zaniedbaniem specjalnych, dodatkowych kosztów akcji reprodukcji i ataku w postaci:

$$P_{max} \cdot c = f_{quantity \text{ spawn}} \quad (1)$$

Po lewej stronie znajduje się ilość energii, która opuszcza środowisko, a po prawej to ile jej przybywa. P_{max} to maksymalna podtrzymywalna w tym środowisku populacja, c to koszt ponoszony przez każdego agenta, a $f_{quantity \text{ spawn}}$ to liczba jedzenia pojawiająca się na mapie w każdej iteracji. Oznacza to że populacja po odpowiednio długim czasie działania symulacji musi znajdować się poniżej tej wartości.

Przebieg jest spokojny, a akcja ataku możliwa do dostrzeżenia jest sporadycznie. Stałe w czasie warunki środowiska nie wymuszają na agentach żadnej formy adaptowania się. Tanią i dobrą strategią przetrwania w takim środowisku jest chodzenie w jednym kierunku, jedzenie kiedy jest to możliwe i rozmnażanie.

2.1.5 Koniec etapu I

Głównym powodem tak dokładnego opisanie tego etapu jest fakt, że niepożądane zjawisko zaobserwowane w trakcie symulacji nie było wynikiem błędu implementacji, ale architektury środowiska i praw nim rządzących. Fakt znalezienia przez agentów sposobu na eksploataowanie reguł symulacji, zespół potraktował jako potwierdzenie poprawności aspektów technicznych implementacji.

2.2 Etap II - Zmienne środowisko oraz dodatkowy surowiec - woda

W tym etapie nie zaobserwowano żadnych nieoczekiwanych przebiegów symulacji, w związku z czym będzie się on skupiał na otrzymanych wynikach. Główne różnice w symulacji to:

1. **Dodatkowy surowiec:** WATER - mechanika pojawiania się i konsumowania przez agentów analogiczna jak w przypadku FOOD
2. Każdy agent ma wewnętrzny poziom energii oraz wody, jeśli którykolwiek spada do 0.0, agent ginie
3. Śmierć agenta spowodowana atakiem na niego skutkuje pojawieniem się w jego miejsce zasobów FOOD i WATER
4. **Pory roku:** intensywność pasywnego pojawiania się pożywienia zamiast wartości stałej jest determinowana przez $A \cdot \sin(\omega t) + B$, co powoduje powstawanie okresów ubogich lub obfitych w pożywienie obydwu typów
5. **Koncentracja surowców:** Używając wielomianów i funkcji trygonometrycznych na mapie wykreślone zostały krzywe wokół których pojawia się 90% zasobów. Obszary wzdłuż krzywych nazywamy **rzekami**. Skrzyżowania rzek nazywamy **oazami**.

Po pomyślnej implementacji wystartowano kilka przebiegów. Najdłuższy z nich był zaplanowany na 200000 iteracji. Łącznie, 3 poniższe przebiegi wygenerowały 6.0 GB danych, z których stworzono wizualizacje. Przebiegi:

1. **10k** iteracji, **4** rzeki, 1.2 GB danych, [link do wizualizacji](#)
2. **50k** iteracji, **2** rzeki, 2.6 GB danych, [link do wizualizacji](#)
3. **200k** iteracji, **2** rzeki, 2.2 GB danych, [link do wizualizacji](#)

W folderach z wizualizacjami znajdują się kształty rzek oraz wykresy przedstawiające częstości wybierania danych akcji.

Za najciekawsze uznano środowisko z dwoma rzekami każdego zasobu. Tworzą się w nim wyraźne 3 oazy, w miejscu których obecne są proporcjonalne do rozmiaru oazy skupiska osobników. Ze względu na długi czas trwania trzeciego przebiegu, zapis musiał być rzadki. Dla przebiegu 3 wykreślone zostają częstości akcji dostępne w folderze. Wizualizacje wszystkich przebiegów posiadają wspólny mianownik w postaci początkowo spokojnego przeżywania w obszarach oaz. W przebiegu 2 i 3, można zauważyć, że populacja wraz z czasem trwania symulacji coraz agresywniej próbuje eksplorować obszar poza oazami, który mimo tego że jest pozornie pełny jedzenia to

ze względu na jego wolną regenerację poza rzekami (poza rzekami pojawia się zaledwie 10% wszystkich zasobów) obszary daleko od oaz są wymagające w przetrwaniu i wymagają koordynowania akcji chodzenia, jedzenia, rozmnażania i niezbliżania się do skupisk innych osobników. Najciekawszym wynikiem **Etapu II** jest zaobserwowanie, że po dostatecznie długim czasie symulacji początkowo statyczna populacja osobników wykształca umiejętność przemieszczania się i zaczyna eksploatować całą mapę, co efektywnie prowadzi do zwiększenia ilości dostępnych zasobów i zbiega się w czasie ze wzrostem liczebności populacji.

3 Uwagi techniczne i implementacyjne

Ze względu na fakt, że symulacje takie jak ta wymagają dużej skali zarówno w sensie liczby osobników jak i w czasie, zdecydowaliśmy, że projekt ten zostanie napisany w języku CUDA, co umożliwi obliczenia na karcie graficznej. Takie rozwiązanie, obok oczywistych zalet wydajnościowych, ma jednak kilka wad.

3.1 Ograniczenia struktury

Symulacja ewolucji jest mało ustrukturyzowana, liczba osobników zmienia się dynamicznie, zaś karty graficzne lubią dane o z góry zadanej liczbie wymiarów. Powoduje to, że pewne wartości (takie jak liczba akcji, liczba sensorów czy kształt wewnętrznej sieci neuronowej) nie podlegają ewolucji (mutacji). Prawdopodobnie symulacja byłaby w stanie dostarczyć dużo więcej ciekawych wyników, gdyby zamiast zwykłej gęstej sieci neuronowej znajdował się w jej miejscu DAG podlegający ewolucji zgodnie z algorytmem NEAT (bądź jakąś jego adaptacją).

3.2 RNG

Kolejnym problemem jest CUDA scheduling. Załóżmy, że w pewnym momencie symulacji na pewnym polu z jedzeniem (lub wodą) dwa lub więcej osobniki wykonały akcję spożywania. Dość prosto można zapewnić, że tylko jeden z nich faktycznie otrzyma to jedzenie lub wodę (za pomocą operacji atomowych), jednakże ze względu na to, że to CUDA scheduling ustala kolejność wykonywania kodu, to nie mamy żadnej kontroli nad tym, który z tych osobników faktycznie otrzyma zasób. Oznacza to, że nie posiadając kontroli nad CUDA scheduling, nie jesteśmy w stanie zapewnić pełnej odtwarzalności eksperymentów. Nie przeszkadza to w odtwarzaniu wielkoskalowych tendencji całej populacji, na obserwacji których najbardziej nam zależy.

4 Podsumowanie

W wyniku symulacji ewolucji otwartej udało nam się zaobserwować wiele interesujących zjawisk, takich jak budowa „farm”, kumulowanie się populacji w „oazach” a także stopniową adaptację części populacji do mniej korzystnych warunków. Dzięki wykorzystaniu języka CUDA byliśmy w stanie przeprowadzić symulację w zadowalająco dużej skali, zarówno populacyjnej jak i czasowej. W trakcie pracy nad projektem pojawiły się pomysły naturalnej rozbudowy reguł symulacji o kolejne, bardziej złożone zjawiska.